

NumPy Revision Guide

Concepts, Examples & 39 Interview Questions — All in One Place

This guide compiles the core NumPy concepts (array creation, attributes, operations, indexing, reshaping, stacking, copies vs. views, linear algebra, masked arrays) alongside commonly asked interview questions — each with runnable code and expected output — for quick, thorough revision.

Table of Contents

Part A — Core Concepts (with examples)

1. Creating Arrays (1D/2D/3D)
2. Array Attributes
3. Placeholder Arrays (zeros, ones, full, empty)
4. `arange()` and `linspace()`
5. Basic Arithmetic & Dot Product
6. Random Number Generation
7. Universal Functions & Aggregation
8. Indexing, Slicing & Iterating
9. `flatten()` vs `ravel()`
10. Reshaping vs Resizing
11. Stacking & Splitting
12. Copies vs Views vs Assignment
13. Sorting & Concatenation
14. Boolean Indexing, Broadcasting & Axis Aggregation
15. Unique Elements, Transpose & Flip
16. Linear Algebra & Masked Arrays

Part B — 39 Interview Questions with Answers & Examples

Part A — Core Concepts

NumPy (Numerical Python) is the foundational library for numerical and scientific computing in Python. Its central object is the **ndarray** — a homogeneous, multi-dimensional array optimized for fast, vectorized operations. Once created, an array's total size can't change, and it must be "rectangular" (not jagged) — every row of a 2D array has the same number of columns. These restrictions let NumPy store data contiguously in memory, making it far faster and more memory-efficient than plain Python lists.

1. Creating Arrays (1D / 2D / 3D)

Arrays are built with **np.array()** from a Python sequence, or generated directly with functions like **np.arange()**.

```
import numpy as np

# 1D array
a = np.array([1, 2, 3, 4, 5])
print(a)

# 2D array
a2 = np.arange(15).reshape(3, 5)
print(a2)
print(type(a2))

# 3D array
a3 = np.arange(24).reshape(2, 3, 4)
print(a3.shape)
```

Output:

```
[1 2 3 4 5]
[[ 0  1  2  3  4]
 [ 5  6  7  8  9]
 [10 11 12 13 14]]
<class 'numpy.ndarray'>
(2, 3, 4)
```

2. Array Attributes

Every ndarray carries metadata describing its structure and memory layout:

Attribute	Meaning
ndim	Number of dimensions/axes
shape	Tuple giving the length along each axis
size	Total number of elements (product of shape)
dtype	Data type of the elements
itemsize	Size in bytes of each element
nbytes	Total bytes consumed by the array's data
strides	Bytes to skip to move to the next element along each axis

```
a = np.arange(15).reshape(3, 5)
print(a.ndim, a.shape, a.size, a.dtype, a.itemsize, a.nbytes)
```

Output:

```
2 (3, 5) 15 int64 8 120
```

3. Placeholder Arrays (zeros, ones, full, empty)

When actual data isn't known up front, NumPy provides placeholder constructors — and their `*_like()` variants, which copy the shape/dtype of an existing array.

```
np.zeros((2, 5))           # all zeros
np.ones((2, 5))           # all ones
np.full((2, 5), 7)        # all elements = 7
np.empty((2, 5))          # uninitialized memory (fast, values are garbage)
np.zeros_like(arr)        # zeros with same shape/dtype as arr
np.full_like(arr, 5)      # 5's with same shape/dtype as arr
```

4. arange() and linspace()

np.arange(start, stop, step) mirrors Python's `range()` but with float steps you can't always predict exactly how many elements you'll get. **np.linspace(start, stop, num)** solves this — you specify the exact number of evenly spaced points instead of a step size.

```
np.arange(10, 30, 5)      # array([10, 15, 20, 25])
np.arange(0, 2, 0.33)     # unpredictable element count with float step
np.linspace(0, 2, 9)      # exactly 9 evenly spaced values between 0 and 2
```

5. Basic Arithmetic & Dot Product

Standard operators apply **element-wise**. Matrix multiplication requires **.dot()** or the `@` operator — not `*`, which multiplies element-wise.

```
a = np.array([10, 20, 30, 40])
b = np.array([1, 2, 3, 4])
print(a - b, a + b, a * b, a / b)

m1 = np.array([[1, 2], [3, 4]])
m2 = np.array([[5, 6], [7, 8]])
print(m1.dot(m2))      # or: m1 @ m2
```

Output:

```
[ 9 18 27 36] [11 22 33 44] [10 40 90 160] [10. 10. 10. 10.]
[[19 22]
 [43 50]]
```

6. Random Number Generation

Modern NumPy favors **np.random.default_rng(seed)** — a Generator object giving reproducible, independent streams — over the older global functions.

```
rg = np.random.default_rng(1)
rg.random((2, 3))         # uniform floats in [0, 1)
np.random.rand(2, 5)      # legacy uniform floats
np.random.randint(0, 10, (3, 4)) # random integers in [0, 10)
np.random.seed(42)        # legacy reproducibility
```

7. Universal Functions & Aggregation

Ufuncs (np.exp, np.sqrt, np.sin, ...) apply element-wise. Aggregations (np.min, np.max, np.sum, np.cumsum) reduce an array to summary statistics, optionally along a chosen axis.

```
a = np.random.randint(0, 10, (3, 4))
np.exp(a); np.sqrt(a); np.sin(a)
np.min(a); np.max(a); a.sum(); a.cumsum()
```

8. Indexing, Slicing & Iterating

Python-style slicing extends naturally to multiple axes, e.g. **a[2:, 2:, 1:]**. Iterating over a multi-dimensional array walks its first axis; use **.flat** to iterate every scalar element regardless of dimensionality.

```
a = np.arange(27).reshape(3, 3, 3)
print(a[0, 0])          # first row of first 2D slice
print(a[2:])            # everything from index 2 onward on axis 0
print(a[2:, 2:, 1:])    # sliced on all three axes

for x in a:              # iterates over axis 0 (2D slices)
    print(x)

for element in a.flat:   # iterates over every scalar element
    print(element)
```

9. flatten() vs ravel()

Feature	flatten()	ravel()
Returns	Independent copy	View (shares memory) when possible
Effect of edits	Original array unaffected	May modify the original array
Speed	Slower (copies data)	Faster (no copy needed)

```
arr = np.array([[1, 2], [3, 4]])
r = arr.ravel(); r[0] = 100
print(arr)          # [[100  2] [3 4]]  <- original changed!

f = arr.flatten(); f[0] = -1
print(arr)          # unchanged by editing f
```

10. Reshaping vs Resizing

Feature	reshape()	resize()
Returns	New view/copy (new object)	None — modifies in place
Original array	Untouched	Changed directly
Element count	Must match exactly	Can pad with zeros or truncate

```
a = np.arange(12).reshape(3, 4)
a.reshape(2, 6)          # new array, same data
a.reshape(6, -1)         # -1 lets NumPy infer that dimension
a.resize(2, 6)           # modifies 'a' itself, in place
```

11. Stacking & Splitting

np.vstack() stacks row-wise (axis 0); **np.hstack()** stacks column-wise (axis 1); **np.column_stack()** behaves like **hstack** but treats 1D arrays as columns. **np.hsplit(a, n)** splits into n equal column groups, or splits after specific column indices when given a tuple.

```
a = np.floor(10 * rg.random((3, 4)))
b = np.floor(10 * rg.random((3, 4)))
np.vstack((a, b))           # stacked as more rows
np.hstack((a, b))           # stacked as more columns
np.column_stack((a, b))

c = np.floor(10 * rg.random((2, 12)))
np.hsplit(c, 3)              # 3 equal pieces
np.hsplit(c, (3, 4))        # split after column 3 and column 4
```

12. Copies vs Views vs Assignment

Method	New object?	Shares data?	Changes propagate?
b = a (assignment)	No — same object	Yes	Yes, both ways
b = a.view()	Yes	Yes (same buffer)	Yes, both ways
b = a.copy()	Yes	No (new buffer)	No — fully independent

```
a = np.array([[0,1,2,3],[4,5,6,7],[8,9,10,11]])
b = a                # same object -> b[1,0]=3 also changes a
c = a.view()         # new object, shared data -> c[1,0]=3 also changes a
d = a.copy()         # independent -> d[1,0]=3 leaves a untouched
```

13. Sorting & Concatenation

```
a = np.floor(100 * rg.random((4, 5)))
np.sort(a)            # each row sorted independently

x = np.array([[1,2,3,4],[5,6,7,8]])
y = np.array([[9,10,11,12],[13,14,15,16]])
np.concatenate((x, y)) # default axis=0 (rows)
np.concatenate((x, y), axis=1) # joins columns instead
```

14. Boolean Indexing, Broadcasting & Axis Aggregation

Comparisons create boolean masks usable directly as indices. Broadcasting lets arrays of different (compatible) shapes combine without explicit copying. The **axis** parameter controls whether an aggregation reduces down columns (axis=0) or across rows (axis=1).

```
a = np.array([[1,2,3,4],[5,6,7,8],[9,10,11,12]])
print(a[a < 5])          # boolean indexing
print(np.nonzero(a < 5))  # indices where condition holds

data = np.array([1.0, 2.0])
print(data * 1.6)        # broadcasting a scalar

grid = np.array([[1,2],[5,3],[4,6]])
print(grid.max(axis=0))   # max of each column
print(grid.max(axis=1))   # max of each row
```

15. Unique Elements, Transpose & Flip

```

a = np.array([11,11,12,13,14,15,16,17,12,13,11,14,18,19,20])
np.unique(a)                                # sorted unique values
np.unique(a_2d, axis=0)                     # unique rows
np.unique(a_2d, axis=0, return_counts=True, return_index=True)

arr_2d.T                                   # transpose: shape (x,y) -> (y,x)
np.flip(arr_2d)                             # reverse all elements
np.flip(arr_2d, axis=0)                     # flip rows (upside-down)
np.flip(arr_2d, axis=1)                     # flip columns (left-right)

```

16. Linear Algebra & Masked Arrays

numpy.linalg.inv(A) computes a square matrix's inverse; **numpy.linalg.det(A)** computes its determinant. A **masked array** pairs data with a boolean mask marking invalid/missing entries, so statistics like **.mean()** automatically skip masked values.

```

A = np.array([[4, 7], [2, 6]])
A_inv = np.linalg.inv(A)
det_A = np.linalg.det(np.array([[1, 2], [3, 4]]))

import numpy.ma as ma
data = np.array([1, 2, -999, 4, 5])
masked = ma.masked_equal(data, -999)
print(masked.mean())    # 3.0 -- ignores the masked value

```

Output:

```
3.0
```

Part B — 39 Interview Questions with Answers

The following questions are the ones most frequently asked about NumPy in technical interviews. Each answer is intentionally concise — cross-reference the section numbers back to Part A for full examples.

Q1. What is NumPy, and how do you create an array?

NumPy is a library for fast numerical computing centered on the `ndarray`. Arrays are created with `np.array()`, or generator functions: `np.zeros()`, `np.ones()`, `np.full()`, `np.arange()`, `np.linspace()`.

Q2. What are the main features of NumPy?

Speed and memory efficiency, a large set of mathematical functions, broadcasting, tight integration with the wider scientific Python stack, native multi-dimensional arrays, rich indexing/slicing, and fine control over memory.

Q3. How do you calculate the dot product of two arrays?

Use `a.dot(b)`, `np.dot(a, b)`, or the `@` operator — all three return the same scalar/matrix product.

Q4. Difference between shallow copy and deep copy?

A shallow copy (`view()`/assignment) shares the same data buffer, so edits propagate both ways. A deep copy (`copy()`) allocates independent memory, so the two arrays never affect each other.

Q5. How do you reshape a NumPy array?

`array.reshape(new_shape)` or `np.reshape(array, new_shape)` — both require the same total element count.

Q6. How do you perform element-wise operations?

Use ordinary arithmetic operators (`+` `-` `*` `/`) directly on same-shaped arrays; NumPy applies them element-by-element automatically. `np.power()` raises elements to a power.

Q7. How do you generate random numbers with NumPy?

`np.random.rand()` for uniform floats, `np.random.randint()` for integers, `np.random.randn()` for standard-normal samples, and `np.random.seed()` (or `default_rng(seed)`) for reproducibility.

Q8. How do you create a NumPy array from a Python list?

Pass the list to `np.array()`, e.g. `np.array([1, 2, 3, 4, 5])`.

Q9. How do you access elements based on a condition?

Boolean indexing: a condition like `arr > 3` produces a boolean array; `arr[arr > 3]` returns only the matching elements.

Q10. What data types does NumPy support?

Common ones include `int`, `float`, `complex`, `bool`, `object`, and `datetime`, among others.

Q11. How do you concatenate two arrays vertically?

`np.vstack((a, b))` or `np.concatenate((a, b), axis=0)`.

Q12. What is matrix inversion in NumPy?

Finding A^{-1} such that $A @ A^{-1}$ equals the identity matrix, computed via `numpy.linalg.inv(A)`.

Q13. Define var() and mean().

`np.var(a, axis=None)` computes variance (spread around the mean); `np.mean(a, axis=None)` computes the arithmetic average. Both can operate along a specific axis.

Q14. How do you convert a multi-dimensional array to 1D?

`array.flatten()` (always a copy) or `array.ravel()` (a view when possible, sharing memory with the original).

Q15. How can you identify outliers in an array?

Either flag values further than k standard deviations from the mean, or use the IQR rule: values below $Q1 - 1.5*IQR$ or above $Q3 + 1.5*IQR$, computed with `np.percentile()`.

Q16. How do you remove missing/null values?

Build a mask with `np.isnan(arr)` and index with its inverse `arr[~mask]`, or use `np.ma.masked_invalid(arr).compressed()`.

Q17. Difference between slicing and indexing?

Slicing (`arr[1:5]`) extracts a contiguous range and typically returns a view. Indexing (`arr[2]`, `arr[[1,3,5]]`) selects specific, possibly non-contiguous elements.

Q18. How do you create an array filled with the same value?

`np.full(shape, value)`, broadcasting like `7 * np.ones(5)`, or a list comprehension converted with `np.array()`.

Q19. What is a masked array?

An array paired with a boolean mask marking certain entries invalid, so aggregate functions ignore masked positions without altering the underlying data.

Q20. What is broadcasting?

NumPy's rule set for combining arrays of different but compatible shapes without physically duplicating data — the smaller array is conceptually stretched to match the larger one.

Q21. How do you sort ascending or descending?

`np.sort(arr)` sorts ascending. For descending order, reverse it (`np.sort(arr)[::-1]`) or index with `np.argsort(-arr)`.

Q22. How are NumPy arrays better than Python lists?

Better performance and memory layout, vectorized operations, broadcasting, and genuine multi-dimensional structure, all backed by compiled C code.

Q23. Difference between reshape() and resize()?

`reshape()` returns a new view/copy without touching the original (element counts must match exactly).
`resize()` changes the array in place and can pad with zeros or truncate.

Q24. What are `vstack()` and `hstack()` used for?

`vstack()` stacks arrays along axis 0 (adds rows); `hstack()` stacks along axis 1 (adds columns).

Q25. How do you get eigenvalues and the determinant of a matrix?

`np.linalg.eigvals(matrix)` for eigenvalues; `np.linalg.det(matrix)` for the determinant.

Q26. How do you compare two NumPy arrays?

`(arr1 == arr2).all()` checks element-wise equality reduced to one boolean; `np.array_equal(arr1, arr2)` checks shape and values together.

Q27. How do you compute a QR decomposition?

`numpy.linalg.qr(a, mode='reduced')` factors A into an orthogonal matrix Q and upper-triangular matrix R, so that $A = Q @ R$.

Q28. What is an ndarray?

NumPy's core data structure — a homogeneous, potentially N-dimensional array optimized for fast mathematical operations.

Q29. What is vectorization?

Applying an operation to an entire array at once (e.g. `arr ** 2`) instead of looping in Python — the loop is pushed down into compiled C code, which is much faster.

Q30. Difference between `copy()`, `view()`, and `=` assignment?

`b = a` creates another reference to the same object (no copy at all). `b = a.view()` creates a new object with independent attributes but shared data. `b = a.copy()` creates a fully independent array.

Q31. Difference between shape and size?

shape is a tuple of the length along each axis (e.g. `(3, 4)`); size is a single integer — the total element count (the product of the shape tuple).

Q32. Python sequences vs NumPy arrays vs pandas?

Python lists/tuples allow mixed types but are slow numerically. NumPy arrays are homogeneous, n-dimensional, and fast. pandas Series/DataFrame build on NumPy but add labels, alignment, and easier handling of missing data — better suited to tabular analysis.

Q33. How do you convert a pandas DataFrame to a NumPy array?

Use the `.values` attribute (or the newer `.to_numpy()` method) on the DataFrame.

Q34. How do you reverse a NumPy array?

Slice with a step of -1: `array[::-1]`.

Q35. Why is NumPy faster than Python lists?

Homogeneous, contiguous memory improves CPU cache use; operations are vectorized in C rather than looped in Python; and linear-algebra calls delegate to optimized libraries like BLAS/LAPACK.

Q36. How do you count occurrences of integers in an array?

`np.bincount(arr)` — the value at index `i` is the count of how many times `i` appears (works only for non-negative integers).

Q37. How do you find the max/min of an array?

`np.max(arr) / np.min(arr)` (or `arr.max()/arr.min()`); add `axis=` to reduce along rows or columns instead of the whole array.

Q38. How can slicing/indexing be used for data cleaning?

Boolean indexing can overwrite values matching a condition in place (e.g. `data[data < 0] = 0` to clip negatives), while filtering with a condition (`data[data > 2]`) extracts a clean subset.

Q39. How do you find unique elements in an array?

`np.unique(array)` returns the sorted distinct values, with optional `return_index`, `return_counts`, and `return_inverse` flags for extra metadata.